

Abstraction, Packages & Exception Handling

Abstract Class

Q57. Create an **abstract class** `Compartment` to represent a rail coach. Provide an abstract method in this class:

```
public abstract String notice();
```

Derive the following classes from `Compartment`:

- `FirstClass`
- `Ladies`
- `General`
- `Luggage`

Override the `notice()` method in each derived class to return a notice message suitable for that specific type of compartment.

Create a class `TestCompartment` containing the `main()` method to perform the following tasks:

1. Declare an array of `Compartment` objects of size **10**.
2. Create a compartment object whose type is decided by a **randomly generated integer** in the range **1 to 4**:
 - 1 → `FirstClass`
 - 2 → `Ladies`
 - 3 → `General`
 - 4 → `Luggage`
3. Store the created object in the array.
4. Invoke the `notice()` method for each object in the array and demonstrate **runtime polymorphism**.

Note: Since the object type is determined randomly, the first object may be a `Luggage` compartment, the second a `Ladies` compartment, the third a `FirstClass` compartment, and so on.

Code:**Compartment.java:**

```
Module 3 > J Compartment.java > Compartment
1  abstract class Compartment {
2
3      // Abstract method
4      public abstract String notice();
5
6  }
```

FirstClass.java:

```
Module 3 > J FirstClass.java > FirstClass
1  class FirstClass extends Compartment {
2
3      @Override
4      public String notice() {
5          return "First Class Compartment";
6      }
7
8  }
```

General.java:

```
Module 3 > J General.java > General
1  class General extends Compartment {
2
3      @Override
4      public String notice() {
5          return "General Compartment";
6      }
7
8  }
```

Ladies.java:

```
Module 3 > J Ladies.java > Ladies
1  class Ladies extends Compartment {
2
3      @Override
4      public String notice() {
5          return "Ladies Compartment";
6      }
7
8  }
```

Luggage.java:

```
Module 3 > J Luggage.java > Luggage
1  class Luggage extends Compartment {
2
3      @Override
4      public String notice() {
5          return "Luggage Compartment";
6      }
7
8  }
```

TestCompartment.java:

```
import java.util.Random;
```

```
class TestCompartment {
```

```
    public static void main(String[] args) {
```

```
        // Create an array of Compartment objects
```

```
        Compartment[] compartments = new Compartment[10];
```

```
        Random random = new Random();
```

```
        // Create random compartment objects
```

```
        for (int i = 0; i < compartments.length; i++) {
```

```
            int choice = random.nextInt(4) + 1;
```

```
            switch (choice) {
```

```
                case 1:
```

```
                    compartments[i] = new FirstClass();
```

```
                    break;
```

```
        case 2:
            compartments[i] = new Ladies();
            break;

        case 3:
            compartments[i] = new General();
            break;

        case 4:
            compartments[i] = new Luggage();
            break;
    }
}

// Display notices
System.out.println("Compartment Notices:");
System.out.println("-----");

for (Compartment c : compartments) {
    System.out.println(c.notice());
}
}
```

Output:

```
✓ TERMINAL powershell ⚠ + ▾ 🗑 ...

PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Projects> cd "C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3"
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> javac Compartment.java
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> javac FirstClass.java
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> javac Ladies.java
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> javac General.java
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> javac Luggage.java
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> javac TestCompartment.java
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> java TestCompartment
Compartment Notices:
-----
Luggage Compartment
First Class Compartment
First Class Compartment
General Compartment
General Compartment
First Class Compartment
Luggage Compartment
General Compartment
Luggage Compartment
Luggage Compartment
PS C:\Users\ArkaLaptop\Documents\VSCode Codes\Java Codes\Module 3> 
```

Packages

Q58. Create a package called **test**.

Inside the package, create a class named **Foundation**.

Define four integer variables:

- var1 → private
- var2 → default (no access modifier)
- var3 → protected
- var4 → public

Create another class outside the package, import the Foundation class, and try to access all four variables. Observe which variables are accessible and which are not.

Code:

Foundation.java:

```
Module 3 > test > J Foundation.java > Foundation
1  package test;
2
3  public class Foundation {
4
5      private int var1 = 10;
6      int var2 = 20;
7      protected int var3 = 30;
8      public int var4 = 40;
9
10 }
```

AccessTest.java:

```
Module 3 > J AccessTest.java > AccessTest
1  import test.Foundation;
2
3  public class AccessTest {
4
5      Run | Debug
      public static void main(String[] args) {
6
7          Foundation obj = new Foundation();
8
9          // System.out.println(obj.var1);    // Private
10         // System.out.println(obj.var2);    // Default
11         // System.out.println(obj.var3);    // Protected
12         System.out.println(obj.var4);      // Public
13
14     }
15
16 }
```

Output:

```
OUTPUT  TERMINAL
powershell
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3\test> javac Foundation.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3\test> cd "C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3"
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac AccessTest.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java AccessTest
40
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
```

Q59. Create a class called **Compartment** that represents a ship compartment with the following attributes:

- Height
- Width
- Breadth

Important: This class should **not conflict** with the Compartment class created in the previous abstract class exercise.

To avoid the conflict, create this class in a **new package** called:

com.wipro.automobile.ship

```
Module 3 > com > wipro > automobile > ship > J Compartment.java > Compartment
1  package com.wipro.automobile.ship;
2
3  public class Compartment {
4
5      private int height;
6      private int width;
7      private int breadth;
8
9      public Compartment(int height, int width, int breadth) {
10         this.height = height;
11         this.width = width;
12         this.breadth = breadth;
13     }
14
15     public void displayDetails() {
16         System.out.println("Height : " + height);
17         System.out.println("Width : " + width);
18         System.out.println("Breadth: " + breadth);
19     }
20 }
```

OUTPUT TERMINAL powershell ⚠ + ▾ 🗑 ... | ⌂ ×

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac
com\wipro\automobile\ship\Compartment.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
```

Q60. Create the following package structure:

```
com
├── automobile
│   ├── Vehicle.java
│   ├── twowheeler
│   │   └── Hero.java
│   └── fourwheeler
│       └── Honda.java
```

Requirements:

1. Create an abstract class `Vehicle`.
2. `Vehicle` contains three abstract methods:
 - o `getModelName()`
 - o `getRegistrationNumber()`
 - o `getOwnerName()`
3. Create `Hero` inside the `twowheeler` package.
4. Create `Honda` inside the `fourwheeler` package.
5. Create a test class to test all methods.

Code:

Vehicle.java:

```
Module 3 > com > automobile > J Vehicle.java > Vehicle
1  package com.automobile;
2
3  public abstract class Vehicle {
4
5      public abstract String getModelName();
6
7      public abstract String getRegistrationNumber();
8
9      public abstract String getOwnerName();
10
11 }
```

Hero.java:Module 3 > com > automobile > twowheeler >  Hero.java >  Hero

```
1  package com.automobile.twowheeler;
2
3  import com.automobile.Vehicle;
4
5  public class Hero extends Vehicle {
6
7      @Override
8      public String getModelName() {
9          return "Hero Splendor";
10     }
11
12     @Override
13     public String getRegistrationNumber() {
14         return "WB 34 AB 1234";
15     }
16
17     @Override
18     public String getOwnerName() {
19         return "Arkaprava Roy";
20     }
21
22     public int getSpeed() {
23         return 80;
24     }
25
26     public void radio() {
27         System.out.println(x: "Radio is ON.");
28     }
29
30 }
```

Honda.java:

```
Module 3 > com > automobile > fourwheeler > J Honda.java > Honda
1  package com.automobile.fourwheeler;
2
3  import com.automobile.Vehicle;
4
5  public class Honda extends Vehicle {
6
7      @Override
8      public String getModelName() {
9          return "Honda City";
10     }
11
12     @Override
13     public String getRegistrationNumber() {
14         return "WB 02 CD 5678";
15     }
16
17     @Override
18     public String getOwnerName() {
19         return "Arkaprava Roy";
20     }
21
22     public int getSpeed() {
23         return 120;
24     }
25
26     public void cdplayer() {
27         System.out.println("CD Player is ON.");
28     }
29
30 }
```

TestVehicle.java:

Module 3 > J TestVehicle.java > TestVehicle

```
1  import com.automobile.twowheeler.Hero;  
2  import com.automobile.fourwheeler.Honda;  
3  
4  public class TestVehicle {  
5  
6      Run | Debug  
7      public static void main(String[] args) {  
8  
9          Hero hero = new Hero();  
10  
11          System.out.println(x: "----- Hero Details -----");  
12          System.out.println("Model Name: " + hero.getModelName());  
13          System.out.println("Registration Number: " + hero.getRegistrationNumber());  
14          System.out.println("Owner Name: " + hero.getOwnerName());  
15          System.out.println("Speed: " + hero.getSpeed());  
16          hero.radio();  
17  
18          System.out.println();  
19  
20          Honda honda = new Honda();  
21  
22          System.out.println(x: "----- Honda Details -----");  
23          System.out.println("Model Name: " + honda.getModelName());  
24          System.out.println("Registration Number: " + honda.getRegistrationNumber());  
25          System.out.println("Owner Name: " + honda.getOwnerName());  
26          System.out.println("Speed: " + honda.getSpeed());  
27          honda.cdplayer();  
28      }
```

Output:

✓ TERMINAL

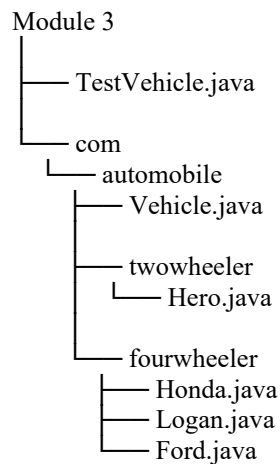
```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac com\automobile\Vehicle.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac com\automobile\twowheeler\Hero.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac com\automobile\fourwheeler\Honda.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
java TestVehicle
Error: Could not find or load main class TestVehicle
Caused by: java.lang.ClassNotFoundException: TestVehicle
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac TestVehicle.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
java TestVehicle
----- Hero Details -----
Model Name: Hero Splendor
Registration Number: WB 34 AB 1234
Owner Name: Arkaprava Roy
Speed: 80
Radio is ON.

----- Honda Details -----
Model Name: Honda City
Registration Number: WB 02 CD 5678
Owner Name: Arkaprava Roy
Speed: 120
CD Player is ON.
```

Q61. Add the following to the previous hands-on:

- Create the `fourwheeler` sub-package under `automobile`.
- Create a `Logan` class extending `Vehicle`.
 - `speed()` → Returns current speed.
 - `gps()` → Controls the GPS device.
- Create a `Ford` class extending `Vehicle`.
 - `speed()` → Returns current speed.
 - `tempControl()` → Controls the air conditioning.
- Create objects of the relevant classes and test them.

Updated Structure



Code:

Logan.java:

```
package com.automobile.fourwheeler;
```

```
import com.automobile.Vehicle;
```

```
public class Logan extends Vehicle {
```

```
    @Override
```

```
    public String getModelName() {
```

```
        return "Mahindra Logan";
```

```
    }
```

```
@Override  
public String getRegistrationNumber() {  
    return "WB 20 XY 4321";  
}  
  
@Override  
public String getOwnerName() {  
    return "Arkaprava Roy";  
}  
  
public int speed() {  
    return 100;  
}  
  
public void gps() {  
    System.out.println("GPS is ON.");  
}  
}
```

Ford.java:

```
package com.automobile.fourwheeler;  
import com.automobile.Vehicle;  
  
public class Ford extends Vehicle {  
  
    @Override  
    public String getModelName() {  
        return "Ford EcoSport";  
    }  
}
```

```
@Override

public String getRegistrationNumber() {

    return "WB 12 EF 9876";

}

@Override

public String getOwnerName() {

    return "Arkaprava Roy";

}

public int speed() {

    return 140;

}

public void tempControl() {

    System.out.println("Air Conditioner is ON.");

}

}
```

Update TestVehicle.java:

```
import com.automobile.twowheeler.Hero;
import com.automobile.fourwheeler.Honda;
import com.automobile.fourwheeler.Logan;
import com.automobile.fourwheeler.Ford;

public class TestVehicle {

    public static void main(String[] args) {
```

```
Hero hero = new Hero();
```

```
System.out.println("----- Hero Details -----");
```

```
System.out.println("Model Name: " + hero.getModelName());
```

```
System.out.println("Registration Number: " + hero.getRegistrationNumber());
```

```
System.out.println("Owner Name: " + hero.getOwnerName());
```

```
System.out.println("Speed: " + hero.getSpeed());
```

```
hero.radio();
```

```
System.out.println();
```

```
Honda honda = new Honda();
```

```
System.out.println("----- Honda Details -----");
```

```
System.out.println("Model Name: " + honda.getModelName());
```

```
System.out.println("Registration Number: " + honda.getRegistrationNumber());
```

```
System.out.println("Owner Name: " + honda.getOwnerName());
```

```
System.out.println("Speed: " + honda.getSpeed());
```

```
honda.cdplayer();
```

```
System.out.println();
```

```
Logan logan = new Logan();
```

```
System.out.println("----- Logan Details -----");
```

```
System.out.println("Model Name: " + logan.getModelName());
```

```
System.out.println("Registration Number: " + logan.getRegistrationNumber());
```

```
System.out.println("Owner Name: " + logan.getOwnerName());
```

```
System.out.println("Speed: " + logan.speed());
```

```
logan.gps();
```

```
System.out.println();
```

```
Ford ford = new Ford();
```

```
System.out.println("----- Ford Details -----");
```

```
System.out.println("Model Name: " + ford.getModelName());
```

```
System.out.println("Registration Number: " + ford.getRegistrationNumber());
```

```
System.out.println("Owner Name: " + ford.getOwnerName());
```

```
System.out.println("Speed: " + ford.speed());
```

```
ford.tempControl();
```

```
}
```

```
}
```

Output:

```
▼ TERMINAL powershell ⚠ + - □ 🗑 ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com\automobile\fourwheeler\Logan.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com\automobile\fourwheeler\Ford.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac TestVehicle.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java TestVehicle

----- Hero Details -----
Model Name: Hero Splendor
Registration Number: WB 34 AB 1234
Owner Name: Arkaprava Roy
Speed: 80
Radio is ON.

----- Honda Details -----
Model Name: Honda City
Registration Number: WB 02 CD 5678
Owner Name: Arkaprava Roy
Speed: 120
CD Player is ON.

----- Logan Details -----
Model Name: Mahindra Logan
Registration Number: WB 20 XY 4321
Owner Name: Arkaprava Roy
Speed: 100
GPS is ON.

----- Ford Details -----
Model Name: Ford EcoSport
Registration Number: WB 12 EF 9876
Owner Name: Arkaprava Roy
Speed: 140
Air Conditioner is ON.
```

Interfaces

Q62. Create:

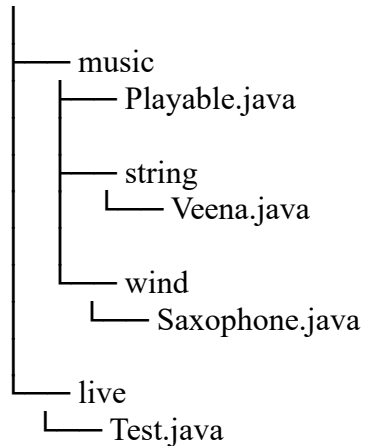
- An interface Playable inside package music
- A class Veena inside package music.string
- A class Saxophone inside package music.wind
- A test class Test inside package live

The Test class should:

1. Create a Veena object and call play()
2. Create a Saxophone object and call play()
3. Store both objects in a variable of type Playable and call play()

Structure:

Module 3



Code:**Playable.java:**

```
Module 3 > music > J Playable.java > Playable
1 package music;
2
3 public interface Playable {
4
5     void play();
6
7 }
```

Veena.java:

```
Module 3 > music > string > J Veena.java > Veena
1 package music.string;
2
3 import music.Playable;
4
5 public class Veena implements Playable {
6
7     @Override
8     public void play() {
9         System.out.println(x: "Playing Veena");
10    }
11
12 }
```

Saxophone.java:

```
Module 3 > music > wind > J Saxophone.java > Saxophone
1 package music.wind;
2
3 import music.Playable;
4
5 public class Saxophone implements Playable {
6
7     @Override
8     public void play() {
9         System.out.println(x: "Playing Saxophone");
10    }
11
12 }
```

Test.java:

```
Module 3 > live > J Test.java > Test
1  package live;
2
3  import music.Playable;
4  import music.string.Veena;
5  import music.wind.Saxophone;
6
7  public class Test {
8
9      Run | Debug
      public static void main(String[] args) {
10
11          // Part (a)
12          Veena veena = new Veena();
13          veena.play();
14
15          // Part (b)
16          Saxophone saxophone = new Saxophone();
17          saxophone.play();
18
19          System.out.println();
20
21          // Part (c)
22          Playable instrument;
23
24          instrument = new Veena();
25          instrument.play();
26
27          instrument = new Saxophone();
28          instrument.play();
29      }
30 }
```

Output:

```
✓ TERMINAL powershell ⚠ + ▢ 🗑 ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac music\Playable.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac music\string\Veena.java

PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac music\wind\Saxophone.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac live\Test.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java live.Test
Playing Veena
Playing Saxophone

Playing Veena
Playing Saxophone
```

Q63. Create:

1. An interface Vehicle with a default method message() that prints:

Inside Vehicle

2. An interface FourWheeler with a default method message() that prints:

Inside FourWheeler

3. A class Car implementing both interfaces.
4. Override message() in Car and call the Vehicle interface's message() using the super keyword.
5. Instantiate Car and call message().

Code:**Vehicle.java:**

```
Module 3 > J Vehicle.java > Vehicle
1 public interface Vehicle {
2
3     default void message() {
4         System.out.println(x: "Inside Vehicle");
5     }
6
7 }
```

FourWheeler.java:

```
Module 3 > J FourWheeler.java > FourWheeler
1 public interface FourWheeler {
2
3     default void message() {
4         System.out.println(x: "Inside FourWheeler");
5     }
6
7 }
```

Car.java:

```
Module 3 > J Car.java > Car
1 public class Car implements Vehicle, FourWheeler {
2
3     @Override
4     public void message() {
5         Vehicle.super.message();
6     }
7
8 }
```

TestCar.java:

```
Module 3 > J TestCar.java > TestCar
1 public class TestCar {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Car car = new Car();
7
8         car.message();
9     }
10
11 }
```

Output:

```
▼ TERMINAL powershell
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac Vehicle.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac FourWheeler.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac Car.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
javac TestCar.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3>
java TestCar
Inside Vehicle
```

Q64. Create an interface named **Test** with one abstract method:

`int myFunction(int a, int b, int c);`

Create two reference variables:

- `t1` → Adds three integers.
- `t2` → Multiplies three integers.

Implement both using **lambda expressions**.

Call `myFunction()` using both references and print the results.

```
Module 3 > J TestLambda.java > TestLambda
1  @FunctionalInterface
2  interface Test {
3
4      int myFunction(int a, int b, int c);
5
6  }
7
8  public class TestLambda {
9
10     Run | Debug
11     public static void main(String[] args) {
12
13         // Lambda expression for addition
14         Test t1 = (a, b, c) -> a + b + c;
15
16         // Lambda expression for multiplication
17         Test t2 = (a, b, c) -> a * b * c;
18
19         System.out.println("Addition = " + t1.myFunction(a: 10, b: 20, c: 30));
20         System.out.println("Multiplication = " + t2.myFunction(a: 10, b: 20, c: 30));
21     }
22 }
23
```

OUTPUT TERMINAL powershell ⚠ + ▾ 🗑 ... | 🔄 ✕

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac TestLambda.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java TestLambda
Addition = 60
Multiplication = 6000
```

Exception Handling

Q65. Write a program that:

1. Takes the size of an array.
2. Reads the array elements.
3. Asks the user for an index.
4. Prints the element at that index.

The program should handle:

- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`

using **exception handling**.

Code:

ExceptionDemo.java:

```
import java.util.Scanner;

public class ExceptionDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            System.out.println("Enter the number of elements in the array");
            int n = Integer.parseInt(sc.nextLine());

            int[] arr = new int[n];
```

```
        System.out.println("Enter the elements in the array");

        for (int i = 0; i < n; i++) {
            arr[i] = Integer.parseInt(sc.nextLine());
        }

        System.out.println("Enter the index of the array element you want to access");

        int index = Integer.parseInt(sc.nextLine());

        System.out.println("The array element at index " + index + " = " + arr[index]);
        System.out.println("The array element successfully accessed");

    }

    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println(e);
    }

    catch (NumberFormatException e) {
        System.out.println(e);
    }

    sc.close();
}

}
```

Output:

```
✓ TERMINAL powershell ⚠ + ▾ 🗑 ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac ExceptionDemo.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java ExceptionDemo
Enter the number of elements in the array:
3
Enter the elements in the array:
20
30
10000
Enter the index of the array element you want to access:
2
The array element at index 2 = 10000
The array element successfully accessed
```

Q66. Write a class **MathOperation** that:

1. Accepts **5 integers through command-line arguments**.
2. Stores them in an array.
3. Finds the **sum** and **average**.
4. Displays the result.
5. Handles possible exceptions:
 - `ArithmeticException`
 - `NumberFormatException`
 - any other exception (`Exception`)

Code:

```
public class MathOperation {  
    public static void main(String[] args) {  
  
        try {  
  
            if (args.length != 5) {  
                throw new Exception("Please enter exactly 5 integers.");  
            }  
  
            int[] arr = new int[5];  
            int sum = 0;  
  
            for (int i = 0; i < 5; i++) {  
                arr[i] = Integer.parseInt(args[i]);  
                sum += arr[i];  
            }  
        }  
    }  
}
```

```
        double average = (double) sum / arr.length;

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);

    }

    catch (ArithmeticException e) {
        System.out.println(e);
    }

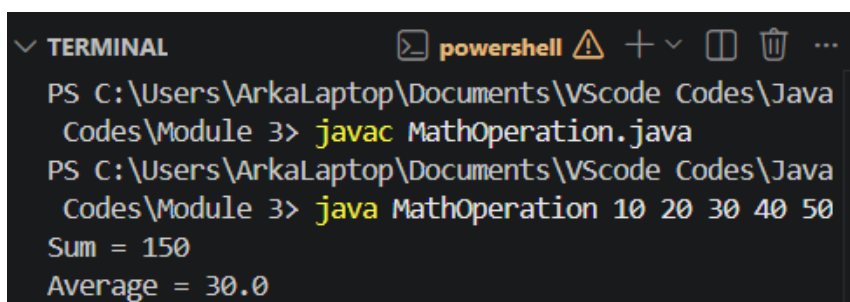
    catch (NumberFormatException e) {
        System.out.println(e);
    }

    catch (Exception e) {
        System.out.println(e.getMessage());
    }

}

}
```

Output:



```

v TERMINAL powershell
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac MathOperation.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java MathOperation 10 20 30 40 50
Sum = 150
Average = 30.0
```

Q67. Write a program that:

1. Takes details of **2 students**.
2. Takes:
 - Student Name
 - Marks in **3 subjects**
3. Calculates the average.
4. Handle:
 - NumberFormatException
 - Negative marks (using your own exception)
 - Marks greater than 100 (using your own exception)

Code:

NegativeValueException.java:

```
Module 3 > J NegativeValueException.java > NegativeValueException
1 class NegativeValueException extends Exception {
2
3     public NegativeValueException(String message) {
4         super(message);
5     }
6
7 }
```

OutOfRangeException.java:

```
Module 3 > J OutOfRangeException.java > OutOfRangeException
1 class OutOfRangeException extends Exception {
2
3     public OutOfRangeException(String message) {
4         super(message);
5     }
6
7 }
```

Student.java:

```
import java.util.Scanner;

public class Student {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            for (int i = 1; i <= 2; i++) {

                System.out.println("Enter the name of Student " + i + ":");
                String name = sc.nextLine();

                int sum = 0;

                System.out.println("Enter marks in 3 subjects:");

                for (int j = 1; j <= 3; j++) {

                    int marks = Integer.parseInt(sc.nextLine());

                    if (marks < 0) {
                        throw new NegativeValueException("Marks cannot be negative.");
                    }

                    if (marks > 100) {
                        throw new OutOfRangeException("Marks should be between 0 and 100.");
                    }
                }
            }
        }
    }
}
```

```
        sum += marks;
    }

    double average = sum / 3.0;

    System.out.println("Student Name : " + name);
    System.out.println("Average Marks : " + average);
    System.out.println();

}

}

catch (NumberFormatException e) {
    System.out.println(e);
}

catch (NegativeValueException e) {
    System.out.println(e.getMessage());
}

catch (OutOfRangeException e) {
    System.out.println(e.getMessage());
}

sc.close();

}

}
```

Output:

```
▼ TERMINAL powershell ⚠ + ▾ 🗑 ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac NegativeValueException.java

PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac OutOfRangeException.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac Student.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java Student
Enter the name of Student 1:
Arka
Enter marks in 3 subjects:
85
90
75
Student Name : Arka
Average Marks : 83.33333333333333

Enter the name of Student 2:
Sayan
Enter marks in 3 subjects:
90
85
70
Student Name : Sayan
Average Marks : 81.66666666666667
```

Q68. Create:

1. A custom exception named InvalidCountryException.
2. Overload its constructors.
3. Create a class UserRegistration.
4. Create a method:

void registerUser(String username, String userCountry)

Rules:

- If userCountry is **not** "India", throw InvalidCountryException.
- Otherwise print:

User registration done successfully.

Code:**InvalidCountryException.java:**

```
Module 3 > InvalidCountryException.java > InvalidCountryException
1  class InvalidCountryException extends Exception {
2
3      public InvalidCountryException() {
4          super();
5      }
6
7      public InvalidCountryException(String message) {
8          super(message);
9      }
10
11 }
```

UserRegistration.java:

```
import java.util.Scanner;

public class UserRegistration {

    public void registerUser(String username, String userCountry)
```

```
        throws InvalidCountryException {

    if (!userCountry.equalsIgnoreCase("India")) {
        throw new InvalidCountryException(
            "User outside India cannot be registered");
    }

    System.out.println("User registration done successfully");
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.println("Enter Username:");
    String username = sc.nextLine();

    System.out.println("Enter Country:");
    String country = sc.nextLine();

    UserRegistration user = new UserRegistration();

    try {
        user.registerUser(username, country);
    }

    catch (InvalidCountryException e) {
        System.out.println(e.getMessage());
    }
}
```

```
        sc.close();  
    }  
}
```

Output:

```
✓ TERMINAL powershell ⚠ + - [ ] [ ] ...  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac InvalidCountryException.java  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac UserRegistration.java  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java UserRegistration  
Enter Username:  
Arka  
Enter Country:  
India  
User registration done successfully  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java UserRegistration  
Enter Username:  
Robert  
Enter Country:  
US  
User outside India cannot be registered
```

Q69. Write a program that:

- Accepts the **name** and **age** from the **command line**.
- Age must satisfy:

$$18 \leq \text{age} < 60$$

Otherwise display an appropriate error message.

Create your own exception class.

Code:

InvalidAgeException.java:

```
Module 3 > J InvalidAgeException.java > InvalidAgeException
1  class InvalidAgeException extends Exception {
2
3      public InvalidAgeException() {
4          super();
5      }
6
7      public InvalidAgeException(String message) {
8          super(message);
9      }
10
11 }
```

Employee.java:

```
public class Employee {

    public static void main(String[] args) {

        try {

            if (args.length != 2) {

                throw new Exception("Please enter Name and Age.");

            }

        }

    }

}
```

```
String name = args[0];

int age = Integer.parseInt(args[1]);

if (age < 18 || age >= 60) {
    throw new InvalidAgeException(
        "Age should be between 18 and 59.");
}

System.out.println("Name : " + name);
System.out.println("Age : " + age);
System.out.println("Employee age is valid.");

}

catch (NumberFormatException e) {
    System.out.println("Age must be a valid integer.");
}

catch (InvalidAgeException e) {
    System.out.println(e.getMessage());
}

catch (Exception e) {
    System.out.println(e.getMessage());
}

}

}
```

Output:

```
✓ TERMINAL powershell ⚠ + ▢ 🗑 ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac InvalidAgeException.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac Employee.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java Employee Arka 21
Name : Arka
Age : 21
Employee age is valid.
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java Employee Sayan 67
Age should be between 18 and 59.
```

Module Mini Project

Mile1 –Student Grade Calculation -- Project Design

Set of student data is available in an array of Student objects. If the given object has any errors, then, the program is expected to return appropriate error messages. Or, if given object has no data errors, then, we need to find the grade and print the same.

Package 1: **com.mile1.bean**

Class Student	
Variables	Description
String name; int marks []; String grade;	Instance variables
Constructors	
public Student () { }	To be Auto generated
public Student (String grade, int [] marks, String name) { // do the initialization }	To be Auto generated
Methods	
Provide public Getters and Setters for all instance variables	To be Auto generated

Package 2: **com. mile1.exception**

All the classes in this package should **extend** the Exception class.

Class	Method	Description
NullMarksArrayException	Override toString() method	Return "mark array is null"
NullNameException	Override toString() method	Return "name is null"
NullStudentObjectException	Override toString() method	Return "object is null"

Package 3: **com.mile1.service**

Class StudentReport	
Method	Description
<pre> public String findGrades (Student studentObject){ // write code here } </pre>	Assumption: Only valid objects are passed to this method. So, just concentrate on the logic part. Get the marks from the given object studentObject. if (any one of the marks is less than 35) then grade is "F"; else do the following: Find the Sum of all the marks. if (sum <150) then grade is "C"; else if (sum <200) then grade is "B"; else if (sum <250) then grade is "A"; else grade is "A+"; return the grade;
<pre> public String validate (Student s) throws NullNameException, NullMarksArrayException, NullStudentObjectException { // write code here } </pre>	If Object passes as parameter itself is null, then, throw the NullStudentObjectException. Else do the following: Check whether there is any null data in the given object. We need to look for null inside the object. 1) If name is null then, throw the NullNameException. 2) Else If marks array is null then throw the NullMarksException 3) Else If all data is valid, return "VALID".

Package3 com. mile1.service

Class StudentService	
Method	Description
<pre> public int findNumberOfNullMarksArray (Student s []) { // write code here } <i>If you are not careful, you will get NullPointerException in this method.</i> </pre>	This method is used to count the number of objects where the marks array is null. Let C=0; Check whether the s is not null. If so, then for all the objects in the s array: if the individual object is not null, then check whether the marks array is null. If so, increase C by 1. Return latest Count value;
<pre> public int findNumberOfNullName (Student s []) { // write code here } <i>Note: If you are not careful, you will get NullPointerException in this method.</i> </pre>	This method is used to count the number of objects where the name is null. // Code like above method // refer findNumberOfNullMarksArray
<pre> public int findNumberOfNullObjects (Student s []) { // write code here } <i>If you are not careful, you will get NullPointerException in this method.</i> </pre>	This method is used to count the number of null objects. Let C=0; Check whether the s is not null. If so, then for all the objects in the s array, if the individual object is null, then increase C by 1. Return latest Count value.

Package 4 com.mile1.main

Class StudentMain
Variables
static Student data [] = new Student [4];
Methods
<pre> StudentMain () { // Constructor for (int i = 0; i < data.length; i++) { data [i] = new Student (); } // initialize the objects data [0] = new Student ("Sekar", new int [] {85,75,95}); data [1] = new Student (null, new int [] {11,22,33}); data [2] = null; data [3] = new Student ("Manoj", null); } </pre>
<pre> public static void main (String [] args): // code as per the following explanation </pre> This main method is used to call the various methods defined in StudentReport class and StudentService class. 1) Call the validate method for all the objects available in data array. 2) If any exception arises during validation, catch the exception and call the printStackTrace () method on that object. 3) If validate method returns "VALID" then, call the findGrades method & print the result returned by findGrades method. Create StudentService Object. Using the above object, Call the findNumberOfNullMarksArray(data) method and print the result. Call the findNumberOfNullName(data) method and print the result. Call the findNumberOfNullObjects(data) method and print the result.

Sample Test Cases which are applied on the above project:

TC1 -- Calculate the grades for **valid** objects -- Check for A+ grade computation.

TC2 -- Calculate the grades for **valid** objects -- Check for F grade computation.

Test for validate method in service package:

Check whether the validate method handles the following situations.

TC3 -- If the Object is null, throw NullStudentObjectException ().

TC4-- If the Name is null, throw NullNameException ().

TC5 -- If the Marks array is null, throw NullMarksArrayException ().

Test for counting methods in service package:

TC6 -- Test findNumberOfNullName function.

TC7 -- Test findNumberOfNullObjects function.

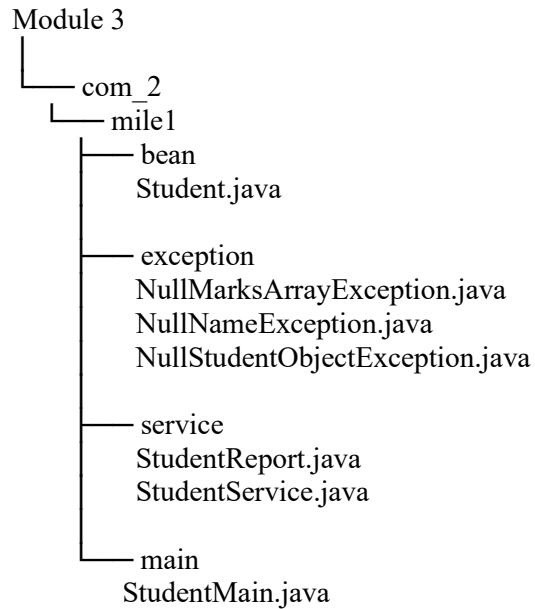
TC8 -- Test findNumberOfNullName

SAMPLE DATA SET1:

Note that your program will be tested with another set of data. Not this set of data.

```
public void init ()    {
    s [0] = new Student ("A1", new int [] {72,73,74});
    s [1] = new Student ("B1", new int [] {75,76,77});
    s [2] = new Student ("C1", new int [] {99,99,99});
    s [3] = new Student ("C3", new int [] {100,100,99});
    s [4] = new Student ("B2", new int [] {13,88,13});
    s [5] = new Student ("C3", new int [] {14,14,99});
    s [6] = new Student ("A2", new int [] {77,55,12});
    s [7] = new Student (null, new int [] {13,88,13});
    s [8] = new Student ("A2", null);
        // this is invalid object -- so no grade calculation
    s [9] = null; // this is invalid object -- so no grade calculation
    expectedgrades =new String [] {"A","A","A+","A+","F","F","F","F"};
}
```

Project Structure:



Code:

Student.java:

```
package com_2.mile1.bean;

public class Student {

    private String name;
    private int[] marks;
    private String grade;

    // Default Constructor
    public Student() {

    }

    // Parameterized Constructor
    public Student(String name, int[] marks) {
        this.name = name;
        this.marks = marks;
    }

    // Getter for Name
    public String getName() {
        return name;
    }

    // Setter for Name
    public void setName(String name) {
```

```
        this.name = name;
    }

    // Getter for Marks
    public int[] getMarks() {
        return marks;
    }

    // Setter for Marks
    public void setMarks(int[] marks) {
        this.marks = marks;
    }

    // Getter for Grade
    public String getGrade() {
        return grade;
    }

    // Setter for Grade
    public void setGrade(String grade) {
        this.grade = grade;
    }
}
```

NullMarksArrayException.java:

```
package com_2.mile1.exception;

public class NullMarksArrayException extends Exception {

    public NullMarksArrayException() {
        super();
    }

    public NullMarksArrayException(String message) {
        super(message);
    }
}
```

NullNameException.java:

```
package com_2.mile1.exception;

public class NullNameException extends Exception {

    public NullNameException() {
        super();
    }

    public NullNameException(String message) {
        super(message);
    }
}
```

```
}  
}
```

NullStudentObjectException.java:

```
package com_2.mile1.exception;  
  
public class NullStudentObjectException extends Exception {  
  
    public NullStudentObjectException() {  
        super();  
    }  
  
    public NullStudentObjectException(String message) {  
        super(message);  
    }  
  
}
```

StudentReport.java

```
package com_2.mile1.service;  
  
import com_2.mile1.bean.Student;  
import com_2.mile1.exception.NullMarksArrayException;  
import com_2.mile1.exception.NullNameException;  
import com_2.mile1.exception.NullStudentObjectException;  
  
public class StudentReport {  
  
    // Method to find the grade  
    public String findGrades(Student student) {  
  
        int sum = 0;  
  
        for (int mark : student.getMarks()) {  
  
            if (mark < 35) {  
                student.setGrade("F");  
                return "F";  
            }  
  
            sum += mark;  
        }  
  
        if (sum < 150) {  
            student.setGrade("C");  
            return "C";  
        } else if (sum < 200) {  
            student.setGrade("B");  
            return "B";  
        } else if (sum < 250) {
```

```
        student.setGrade("A");
        return "A";
    } else {
        student.setGrade("A+");
        return "A+";
    }
}

// Method to validate the student object
public String validate(Student student)
    throws NullStudentObjectException,
           NullNameException,
           NullMarksArrayException {

    if (student == null) {
        throw new NullStudentObjectException("Student object is null");
    }

    if (student.getName() == null) {
        throw new NullNameException("Student name is null");
    }

    if (student.getMarks() == null) {
        throw new NullMarksArrayException("Marks array is null");
    }

    return "VALID";
}
}
```

StudentService.java:

```
package com_2.mile1.service;

import com_2.mile1.bean.Student;

public class StudentService {

    // Count students whose marks array is null
    public int findNumberOfNullMarksArray(Student[] data) {

        int count = 0;

        for (Student student : data) {
            if (student != null && student.getMarks() == null) {
                count++;
            }
        }

        return count;
    }

    // Count students whose name is null
    public int findNumberOfNullName(Student[] data) {
```

```
        int count = 0;

        for (Student student : data) {
            if (student != null && student.getName() == null) {
                count++;
            }
        }

        return count;
    }

    // Count null student objects
    public int findNumberOfNullObjects(Student[] data) {

        int count = 0;

        for (Student student : data) {
            if (student == null) {
                count++;
            }
        }

        return count;
    }
}
```

Output:

```
▼ TERMINAL powershell + - [ ] [ ] ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\bean\Student.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\exception\NullMarksArrayException.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\exception\NullNameException.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\exception\NullStudentObjectException.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\service\StudentReport.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\service\StudentService.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> javac com_2\mile1\main\StudentMain.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 3> java com_2.mile1.main.StudentMain
Sekar Grade : A+
com_2.mile1.exception.NullNameException: Student name is null
    at com_2.mile1.service.StudentReport.validate(StudentReport.java:51)
    at com_2.mile1.main.StudentMain.main(StudentMain.java:27)
com_2.mile1.exception.NullStudentObjectException: Student object is null
    at com_2.mile1.service.StudentReport.validate(StudentReport.java:47)
    at com_2.mile1.main.StudentMain.main(StudentMain.java:27)
com_2.mile1.exception.NullMarksArrayException: Marks array is null
    at com_2.mile1.service.StudentReport.validate(StudentReport.java:55)
    at com_2.mile1.main.StudentMain.main(StudentMain.java:27)

Number of Null Marks Array : 1
Number of Null Names : 1
Number of Null Objects : 1
```